01a Programmiersprachen

01a Programmiersprachen

00 Übersicht · Zurück: 01 Einführung · Weiter: 01b Java

Was ist eine Programmiersprache?

- Erlaubt die Formulierung von **Algorithmen** in einer für Menschen verständlichen Sprache.
- **Präzise genug** (eindeutig), um von Maschinen interpretiert zu werden.
- **Verständlich genug**, damit Menschen sie nachvollziehen können.

```
flowchart LR
    M[Mathematischer Ausdruck] -->|ggf. Umformung| P[Formulierung in formaler Sprache]
    P -->|Compiler: automatisierte Übersetzung| B[Maschinencode / Bytecode]
```

Klassen / Paradigmen

Paradigma	Beispiele	Kernidee
Imperativ / prozedural	Fortran, Cobol, C, Pascal	meist ältere Sprachen (60er–80er), heute selten. Komplexe Algorithmen werden in überschaubare Einheiten (**Prozeduren**) mit klar definierter Schnittstelle aufgeteilt. Fast alle modernen Sprachen unterstützen auch prozedurale Aspekte
Objektorientiert	Java, C++, C#	Weiterentwicklung der prozeduralen Sprachen in den 90ern. Zentral: **Klassen** kapseln Daten (**Attribute**) und Verhalten (**Methoden**)
Funktional	Lisp, Haskell, F#, Scala	Funktionen im Vordergrund. Daten werden als Parameter übergeben und meist nicht nachträglich verändert
Script-Sprachen	JavaScript, Python	unterstützen meist mehrere der obigen Paradigmen

Trend
Hin zu **hybriden Sprachen**, die Konzepte mehrerer Paradigmen unterstützen.

Syntax, Semantik, Pragmatik

Programmiersprachen sind **formale Sprachen** zur Formulierung von Algorithmen.

	Bedeutung	Prüfbarkeit
Syntax	gültige Zeichenmenge (**Alphabet**) und gültige Verknüpfungen zu Ausdrücken (**Grammatik**)	gut automatisiert prüfbar (Compiler)
Semantik	ordnet Ausdrücken eine konkrete **Bedeutung** zu; teils erst im Kontext klar (mehrdeutig)	teilweise: **statisch** (z.B. korrekte Datentypen durch den Compiler) und **dynamisch** (Verhalten zur Laufzeit)
Pragmatik	**adäquate Verwendung** der Ausdrücke für bestimmte Probleme. Ziel: Lesbarkeit und Wartbarkeit. Aus Erfahrung abgeleitet	eingeschränkt, z.B. durch Code Checker

Beispiele

Ebene	Deutsch	Java	Wer merkt es?
Syntax (Zeichen)	„A44Aafährt mit dem Fahrrad“	`int value = @;`	Compiler
Syntax (Grammatik)	„Anna fahren mit dem Fahrrad“	`int value =: 2;`	Compiler
Semantik	„Max dreht am Rad“ (nervös? oder dreht wirklich ein Rad?)	`3 / 0` (Division durch 0 nicht erlaubt)	meist erst zur **Laufzeit**
Pragmatik	„Tom und Harry fahren mit einem vierrädrigen Vehikel“	`int yes = 0; int no = 1;` (ungünstige Datenstruktur für Ja/Nein, besser boolean)	niemand: keine echten Fehler, aber Unschönheiten, die zu Fehlern führen können

Merksatz
Syntax (und teilweise Semantik) prüft der **Compiler**, viele semantische Fehler fallen erst zur **Laufzeit** auf.